



THIRD EDITION

آموزش یونیتی

Unity in action



Unity
IN ACTION



Unity IN ACTION

THIRD EDITION

Multiplatform game development in C#



Joseph Hocking

Foreword by Jesse Schell



Developing graphics for your game



ارتقاء گرافیک بازی

This chapter covers

- Understanding art assets used in game development
- Building prototype levels through whiteboxing
- Using 2D images in Unity
- Importing custom 3D models
- Crafting particle effects



Understanding art assets

- دارایی (Asset) گرافیکی واحدی از اطلاعات بصری (معمولاً یک فایل) است که توسط بازی استفاده می شود. این اصطلاح کلی در مورد تمام محتوای بصری صدق می کند: فایل های تصویری دارایی های هنری هستند، مدل های سه بعدی دارایی های هنری هستند و غیره.
- در واقع، یک دارایی گرافیکی یا هنری یک نوع خاصی از دارایی است، که شما یاد گرفته اید هر فایلی است که توسط بازی استفاده می شود (مانند یک اسکریپت) یک دارایی است از این رو پوشه اصلی Assets در Unity است.

Type of art asset	Definition
2D image	Flat pictures. To make a real-world analogy, 2D images are like paintings and photographs.
3D model	3D virtual objects (almost a synonym for <i>mesh objects</i>). To make a real-world analogy, 3D models are like sculptures.
Material	A packet of information that defines the surface properties of any object that the material is attached to. These surface properties can include color, shininess, and even subtle roughness.
Animation	Packets of information that define the movement of the associated object. These are detailed movement sequences created ahead of time, as opposed to code that calculates positions on the fly.
Particle system	An orderly mechanism for creating and controlling large numbers of small moving objects. Many visual effects, like fire, smoke, or spraying water, are created this way.



مدل سه بعدی: مدل یک شی مجازی سه بعدی است. فصل ۱ اصطلاح شی مش را معرفی کرد و مدل سه بعدی عملاً مترادف آن است. این اصطلاحات اغلب به جای یکدیگر استفاده می شوند، اما شی مش به شدت به هندسه شی سه بعدی (خطوط و اشکال متصل) اشاره دارد، در حالی که مدل کمی مبهم است و اغلب شامل سایر ویژگی های شی است.

DEFINITION A *model* is a 3D virtual object. Chapter 1 introduced the term *mesh object*, and *3D model* is practically a synonym. The terms are frequently used interchangeably, but *mesh object* strictly refers to the geometry of the 3D object (the connected lines and shapes), whereas *model* is a bit more ambiguous and often includes other attributes of the object.

جنس اجسام: یک ماده (*material*) بسته ای از اطلاعات است که ویژگی های سطح (رنگ، درخشندگی و غیره) هر جسمی را که به آن متصل است، تعریف می کند. تعریف ویژگی های سطح به طور جداگانه، چندین شی را قادر می سازد تا یک ماده را به اشتراک بگذارند (مثلاً تمام دیوارهای قلعه).

DEFINITION A *material* is a packet of information that defines the surface properties (color, shininess, and so forth) of any object that it's attached to. Defining surface properties separately enables multiple objects to share a material (all the castle walls, for example).



انیمیشن بسته ای از اطلاعات است که حرکت شی مرتبط را مشخص می کند. از آنجایی که این حرکات را می توان به طور مستقل از خود شی تعریف کرد، می توان آنها را به روشی ترکیبی با چندین شی مورد استفاده قرار داد.

DEFINITION An *animation* is a packet of information that defines the movement of the associated object. Because these movements can be defined independently from the object itself, they can be used in a mix-and-match way with multiple objects.

سیستم ذرات مکانیزمی منظم برای ایجاد و کنترل تعداد زیادی از اجسام متحرک معمولاً کوچک هستند - از این رو ذره نامیده می شود.

DEFINITION A *particle system* is an orderly mechanism for creating and controlling large numbers of moving objects. These moving objects are usually small—hence the name *particle*—but they don't have to be.

ساخت بسیاری از اشیاء گرافیکی بازی در نرم افزارهای خارجی و نه در خود Unity انجام می شود. اما مواد و سیستم های ذرات در Unity ایجاد می شوند، ولی سایر دارایی های گرافیکی با استفاده از نرم افزار خارجی ایجاد می شوند.



اولین موضوع تولید محتوا که به آن می پردازیم جعبه سفید (*whiteboxing*) است. این فرآیند معمولاً اولین گام در ساخت یک مرحله در رایانه (پس از طراحی سطح روی کاغذ) است. همانطور که از نام آن پیداست، دیوارهای صحنه را با هندسه خالی (جعبه های سفید) مسدود می کنید. همانطور که در فصل های قبل مشاهده کردید، این منظره خالی ابتدایی ترین نوع مدل سه بعدی است و پایه ای را برای نمایش تصاویر دو بعدی فراهم می کند.

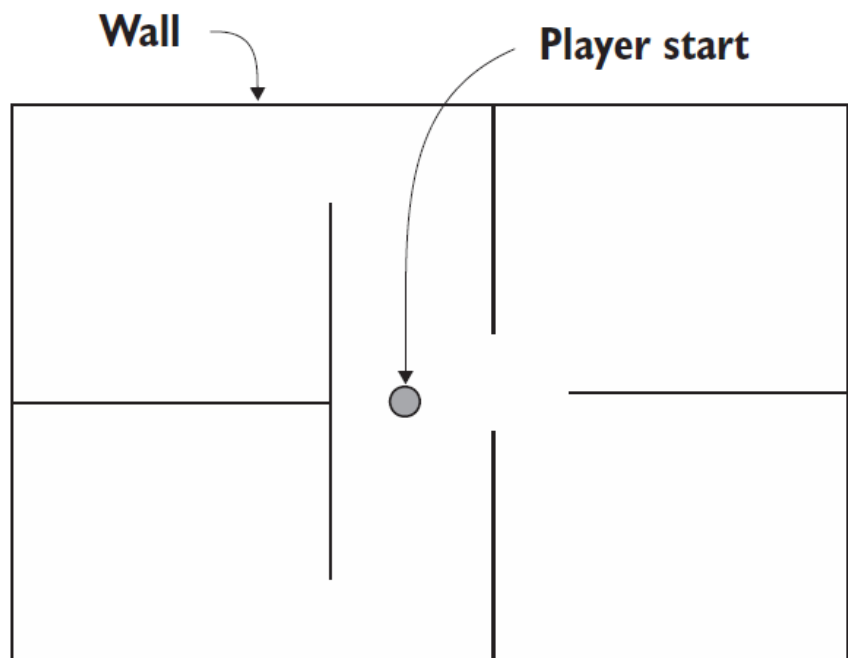
طراحی مرحله، رشته برنامه ریزی و ایجاد صحنه ها (یا سطوح) در بازی است. یک طراح مرحله، یک متخصص طراحی سطح است.

DEFINITION *Level design* is the discipline of planning and creating scenes (or levels) in the game. A *level designer* is a practitioner of level design.

از آنجایی که تیم های توسعه بازی بزرگ تر شده اند و اعضای تیم تخصصی تر شده اند، یک گردش کار رایج برای ایجاد مراحل این است که طراح مرحله یا سطح اولین نسخه از مرحله را از طریق **Whiteboxing** ایجاد کند. سپس این مرحله ساده برای پرداخت بصری به تیم هنری تحویل داده می شود. اما حتی در یک تیم کوچک، که در آن شخص واحدی در حال طراحی مرحله و ایجاد گرافیک برای بازی است، این جریان کاری که ابتدا وایت باکسینگ و سپس صیقل دادن تصاویر را انجام می دهد، عموماً بهترین کار را دارد. بالاخره شما باید از جایی شروع کنید و وایت باکسینگ پایه و اساس روشنی را برای ایجاد تصاویر بصری ایجاد می کند.



هدف دوم وایت باکسینگ این است که مرحله به سرعت به حالت قابل پخش برسد. مرحله بازی ممکن است تمام نشده باشد (در واقع، یک مرحله بعد از وایت باکسینگ تا پایان فاصله زیادی دارد)، اما این نسخه خشن کاربردی است و می تواند از گیم پلی پشتیبانی کند. حداقل، بازیکن می تواند در اطراف صحنه قدم بزند (به نسخه ی نمایشی در فصل ۲ فکر کنید). به این ترتیب، می توانید قبل از صرف زمان و انرژی زیاد برای انجام کارهای دقیق، تست کنید تا مطمئن شوید که سطح به خوبی کنار هم قرار می گیرد (مثلاً آیا اتاق ها برای این بازی اندازه مناسبی دارند؟). اگر چیزی ناقص است (مثلاً متوجه شده اید که فضاها باید بزرگتر باشند)، تغییر و آزمایش مجدد در مرحله وایت باکس بسیار آسان تر است.



ساخت یک مرحله بازی در کامپیوتر به دنبال طراحی مرحله روی کاغذ است. ما قصد نداریم وارد بحث بزرگی در مورد طراحی مرحله شویم. همانطور که فصل ۲ در مورد طراحی بازی اشاره کرد، طراحی مرحله (که زیر مجموعه ای از طراحی بازی است) یک بحث بزرگ است که می تواند یک کتاب کامل را به خودی خود پر کند. برای اهداف خود، می خواهیم یک مرحله پایه را ترسیم کنیم، با طراحی کمی که در طرح وجود دارد، تا به ما هدفی بدهیم که به سمت آن کار کنیم.

Figure 4.1 Floor plan for the level:
four rooms and a central corridor

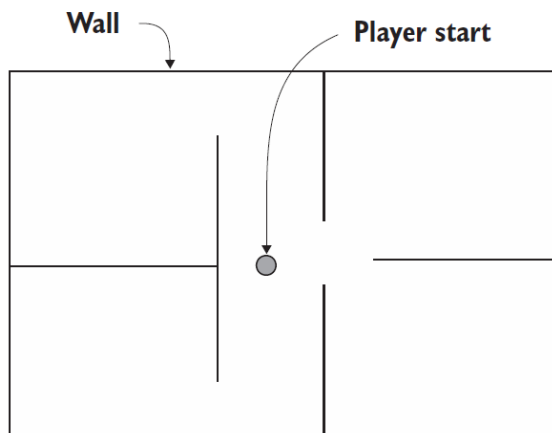


Figure 4.1 Floor plan for the level: four rooms and a central corridor

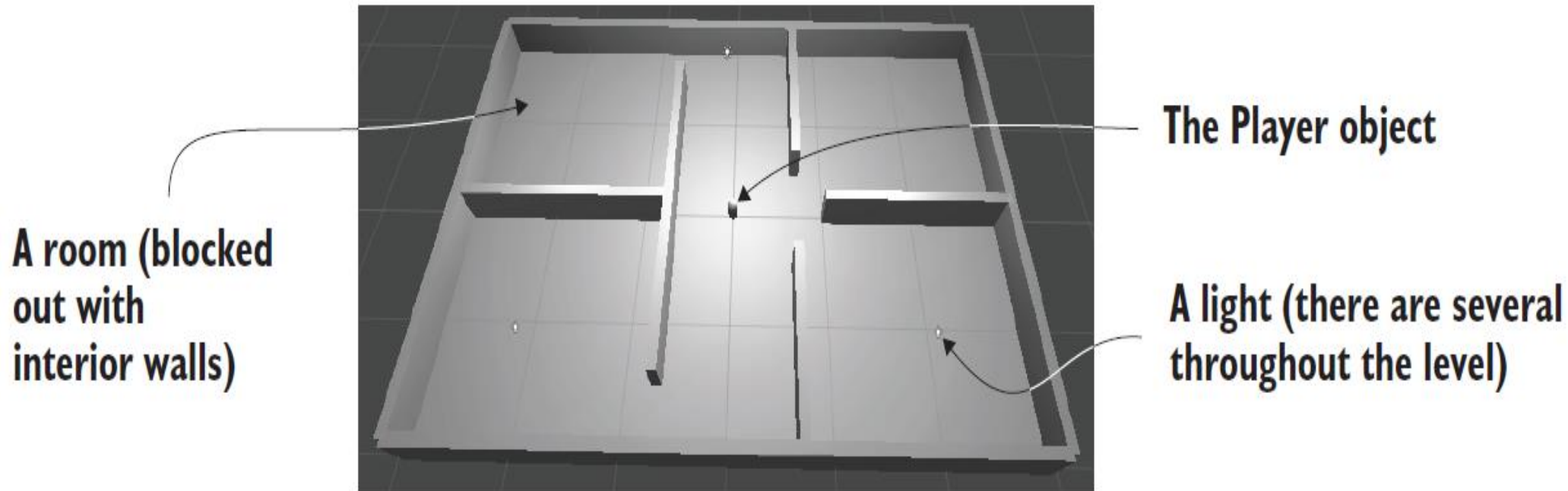


Figure 4.3 Whitebox level of the floor plan in figure 4.1



Texturing the scene with 2D images

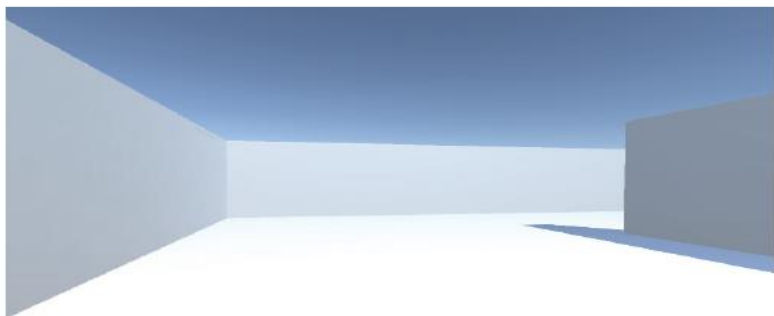
تنظیم طرح و رویه اجسام (بافت) با استفاده از تصاویر دو بعدی

بافت یک تصویر دو بعدی است که برای بهبود گرافیک سه بعدی استفاده می شود .

DEFINITION A *texture* is a 2D image being used to enhance 3D graphics. That's the totality of what the term means; don't confuse yourself by thinking that any of the uses of textures are part of how the term is defined. No matter how the image is being used, it's still referred to as a texture.

- بافت ها در گرافیک سه بعدی کاربردهای متعددی دارند، اما ساده ترین کاربرد آن نمایش روی سطح مدل های سه بعدی است.
- تصاویر ۲ بعدی به عنوان تصویر زمینه پوشش دیوارها عمل می کنند.

Before texturing
(shading only from lights)



After applying textures
(1 floor texture, 1 on all walls)

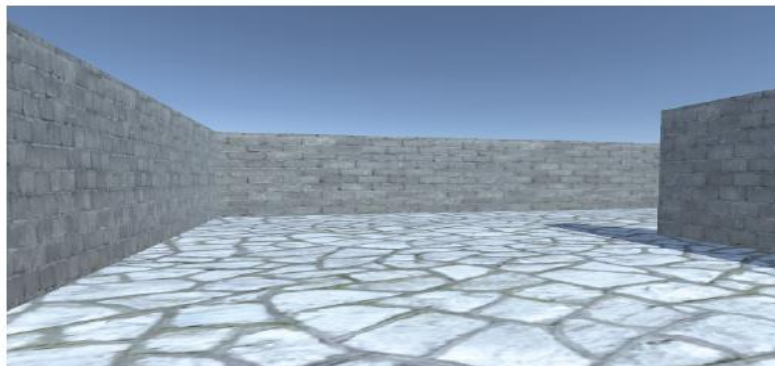


Figure 4.4 Comparing the level before and after textures



یونیتی استفاده از بسیاری از فرمت‌های فایل را پشتیبانی می‌کند، بنابراین می‌توانید یکی از فرمت‌های نشان داده شده در جدول را انتخاب کنید

2D image file formats supported by Unity

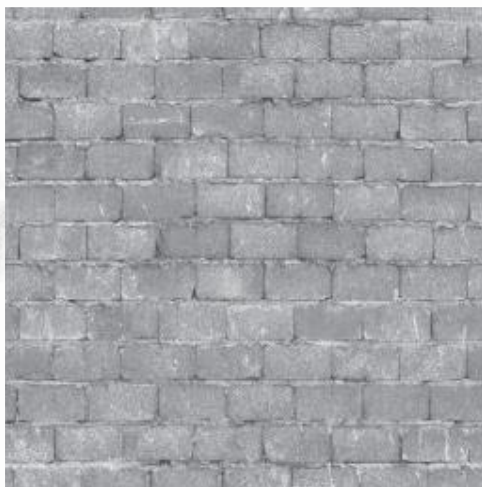
File type	Pros and cons
PNG	Commonly used on the web. Lossless compression; has an alpha channel.
JPG	Commonly used on the web. Lossy compression; no alpha channel.
GIF	Commonly used on the web. Lossy compression; no alpha channel. (Technically, the loss isn't from compression; rather, data is lost when the image is converted to 8-bit. Ultimately, it amounts to the same thing.)
BMP	Default image format on Windows. No compression; no alpha channel.
TGA	Commonly used for 3D graphics; obscure everywhere else. No or lossless compression; has an alpha channel.
TIFF	Commonly used for digital photography and publishing. No or lossless compression; no alpha channel.
PICT	Default image format on old Macs. Lossy compression; no alpha channel.
PSD	Native file format for Adobe Photoshop. No compression; has an alpha channel. The main reason to use this file format would be the advantage of using Photoshop files directly.



اگرچه Unity هر یک از انواع تصویر نشان داده شده در جدول اسلاید قبل را برای وارد کردن و استفاده به عنوان یک بافت می پذیرد، اما فرمت های فایل به طور قابل توجهی در ویژگی هایی که پشتیبانی می کنند متفاوت است. دو عامل به ویژه برای تصاویر دو بعدی وارد شده به عنوان بافت مهم است: تصویر چگونه فشرده می شود و آیا کانال آلفا دارد؟

بین این دو ملاحظه، دو فرمت فایلی که من برای بافت های Unity توصیه می کنم، PNG و TGA هستند. تارگاس (TGA) قبل از اینکه PNG به طور گسترده در اینترنت مورد استفاده قرار گیرد، فرمت فایل مورد علاقه برای بافت گرافیکی سه بعدی بود. این روزها، PNG تقریباً از نظر فناوری مشابه است، اما بسیار گسترده تر است، زیرا هم در وب و هم به عنوان بافت مفید است.

تصاویری که برای سطوح بافت استفاده می شوند معمولاً قابل استفاده بصورت کاشی را دارند به طوری که می توان آنها را در سطوح بزرگ مانند کف تکرار کرد.

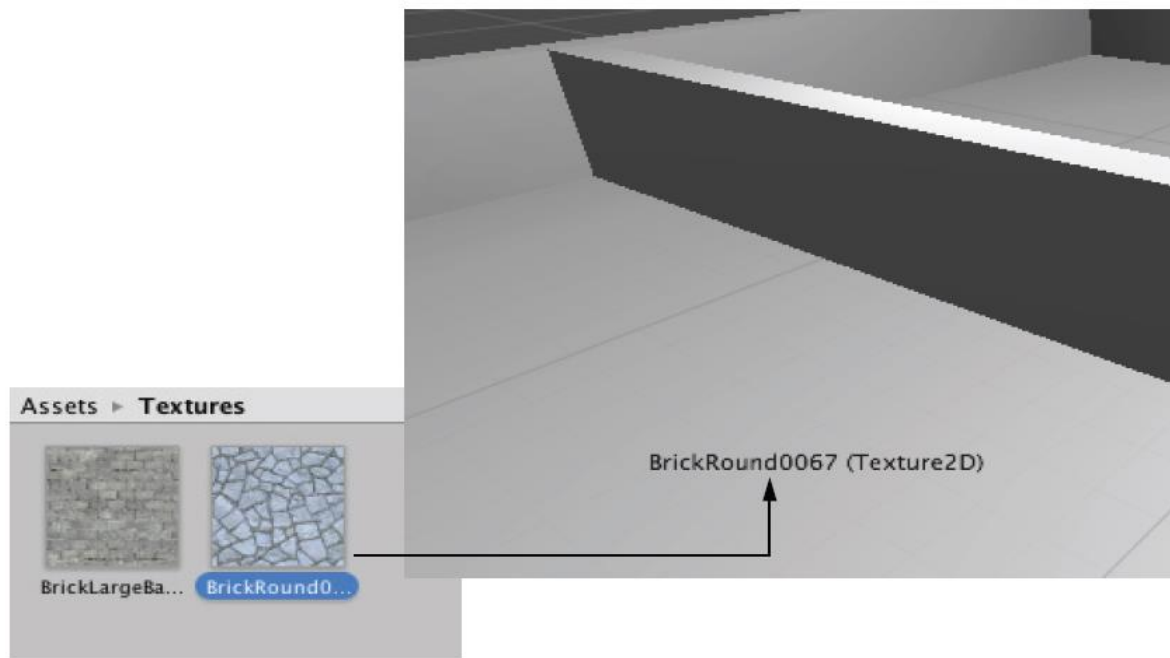


یک تصویر کاشی پذیر (گاهی اوقات به عنوان کاشی بدون درز از آن یاد می شود) تصویری است که در آن لبه های مقابل در کنار هم قرار می گیرند. به این ترتیب، تصویر را می توان بدون هیچ گونه درز قابل مشاهده ای بین تکرارها تکرار کرد. مفهوم تکسچر سه بعدی درست مانند کاغذ دیواری در صفحات وب است.



فایل های تصاویر را از مکانشان در کامپیوتر به نمای پروژه در Unity بکشید. این فایل ها را در پروژه Unity شما کپی می کند، در این مرحله آنها به عنوان بافت وارد می شوند و می توانند در صحنه سه بعدی استفاده شوند. اگر کشیدن فایل روی آن مشکل است، می توانید در عوض روی Project کلیک راست کرده و برای دسترسی به انتخابگر فایل، Import New Asset را انتخاب کنید.

از نظر فنی، بافت ها مستقیماً در هندسه اعمال نمی شوند. در عوض، بافت ها می توانند بخشی از مواد باشند، و مواد در هندسه اعمال می شوند. همانطور که در مقدمه توضیح داده شد، یک ماده مجموعه ای از اطلاعات است که ویژگی های یک سطح را تعریف می کند. این اطلاعات می تواند شامل یک بافت برای نمایش در آن سطح باشد. این غیر جهت قابل توجه است زیرا می توان از یک بافت با چندین متریال استفاده کرد. همانطور که گفته شد، معمولاً هر بافت با متریال متفاوتی همراه است، بنابراین برای راحتی، یونیتی به شما امکان می دهد یک بافت را روی یک شی رها کنید و سپس به طور خودکار یک متریال جدید ایجاد می کند.



اگر یک بافت را از نمای پروژه روی یک شی در صحنه بکشید، Unity یک متریال جدید ایجاد می کند و آن را روی شی اعمال می کند.



Assets > Create > Material

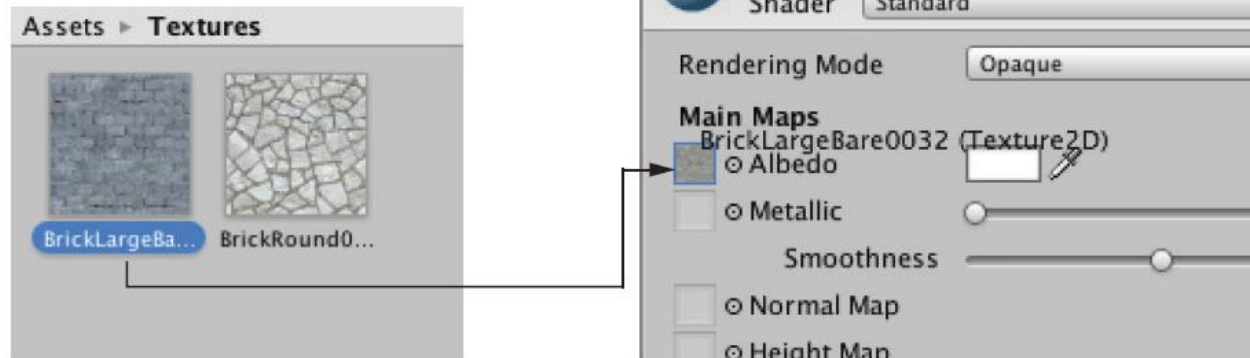


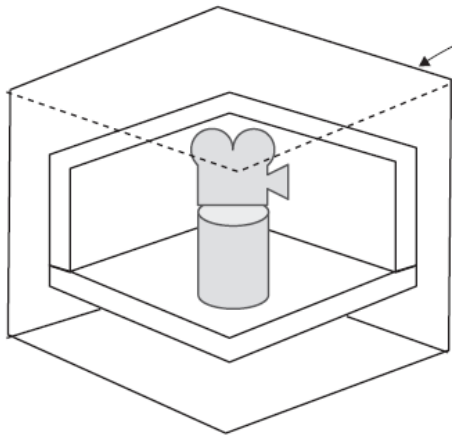
Figure 4.8 Select a material to see it in the Inspector and then drag textures to the material properties.

شما می توانید این ظاهر را با استفاده از خاصیت کاشی کاری متریال تنظیم کنید. متریال را در **Project** انتخاب کنید و سپس شماره کاشی کاری را در **Inspector** تغییر دهید (با مقادیر جداگانه **X** و **Y** برای کاشی کاری در هر جهت). مطمئن شوید که کاشی کاری نقشه اصلی را تنظیم می کنید نه نقشه ثانویه (این ماده به صورت اختیاری از یک نقشه بافت ثانویه برای جلوه های پیشرفته استفاده می کند). کاشی کاری پیش فرض ۱ است (این کاشی کاری نیست، با اینکه تصویر روی کل سطح کشیده شده است). اعداد را به چیزی شبیه ۸ تغییر دهید و ببینید در صحنه چه اتفاقی می افتد. اعداد در هر دو متریال را به کاشی کاری که ظاهر خوبی دارد تغییر دهید.



A **skybox** is a cube surrounding the camera with pictures of the sky on each side. No matter what direction the camera is facing, it's looking at a picture of the sky.

اسکای باکس مکعبی است که دوربین را با تصاویری از آسمان در هر طرف احاطه کرده است. مهم نیست که دوربین در چه جهتی قرار دارد، به تصویری از آسمان نگاه می کند.



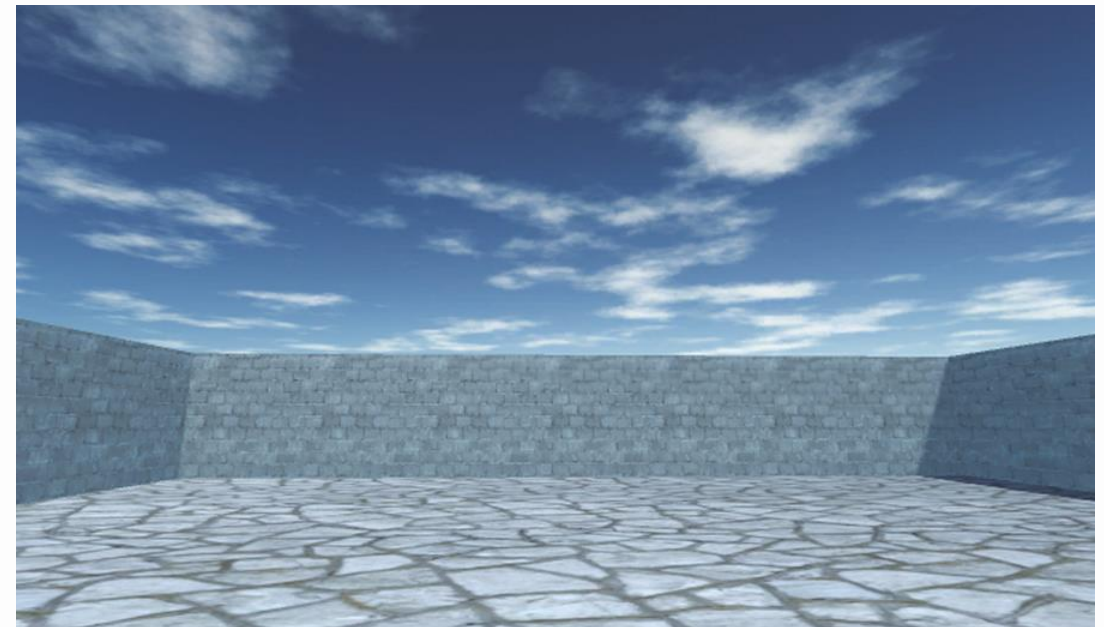
The skybox—functionality needed:

Render behind everything else in the scene.

Stay centered on the camera, so that the sky will seem too far away for the player's movements to affect it.

Full brightness with no shading applied, to avoid any lighting differences between sides of the cube.

Figure 4.9 Diagram of a skybox

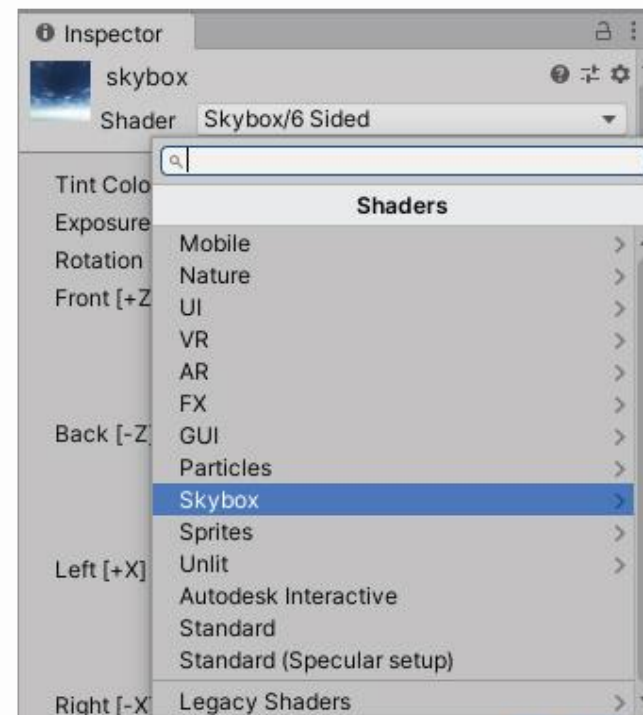


Creating a new skybox material

- First, create a new material
- Next, you need to change the shader used by this material.
- For skyboxes, Unity has a different shader. Select the Skybox section and choose 6 Sided in the submenu.

DEFINITION A shader is a short program that outlines instructions for drawing a surface, including whether to use any textures. The computer uses these instructions to calculate the pixels when rendering the image. The most common shader takes the color of the material and darkens it according to the light, but shaders can also be used for all sorts of visual effects.

سایه زن برنامه کوتاهی است که دستورالعمل‌هایی را برای ترسیم یک سطح، از جمله اینکه آیا از هر بافتی استفاده شود یا خیر، توضیح می‌دهد. کامپیوتر از این دستورالعمل‌ها برای محاسبه پیکسل‌ها هنگام رندر کردن تصویر استفاده می‌کند. رایج‌ترین سایه زن رنگ مواد را می‌گیرد و با توجه به نور آن را تیره می‌کند، اما سایه بان‌ها همچنین می‌توانند برای انواع جلوه‌های بصری استفاده شوند.



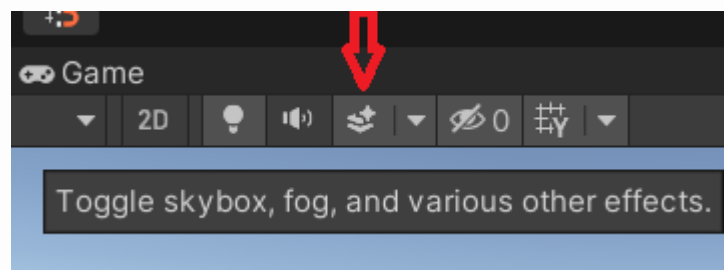
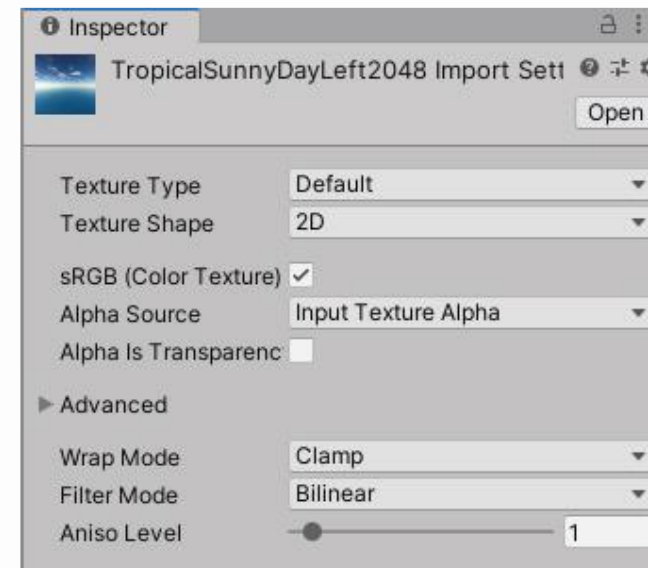


- Import the skybox images into Unity (drag the files into the Project)
- Click the imported texture to see its properties in the Inspector and change the Wrap Mode setting from Repeat to Clamp.

این تنظیم باعث میشود مرزهای اتصال دو تصویر درست به نظر برسد



Figure 4.12 Six images for the sides of a skybox





پس از ساختن متریال آسمان آن را بصورت زیر به تصویر اضافه می کنیم

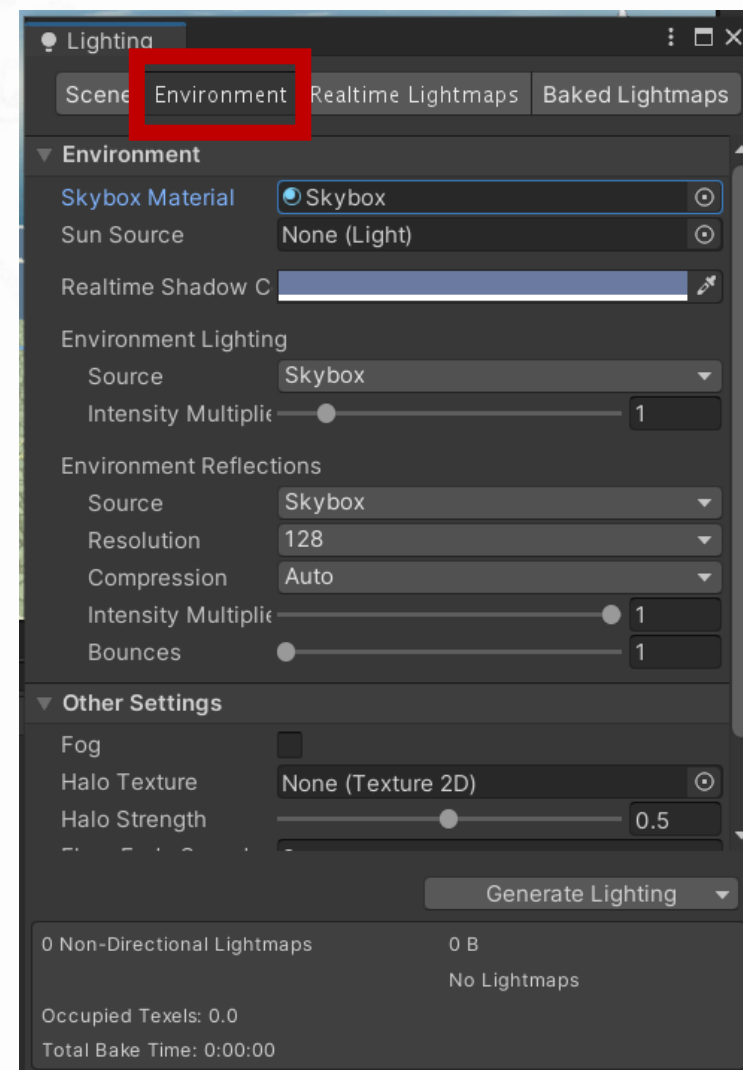
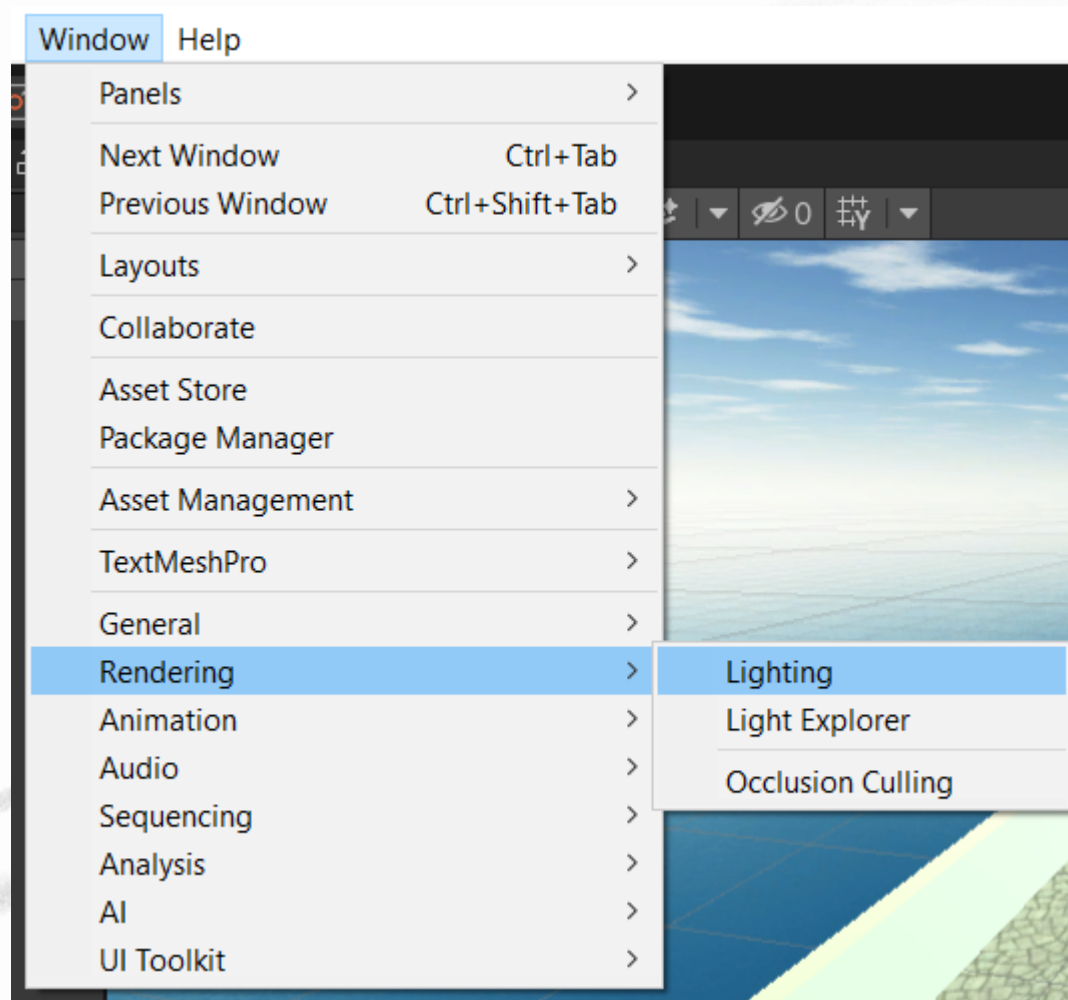


Table 4.3 3D Model file formats supported by Unity

File type	Pros and cons
FBX	Mesh and animation; recommended option when available.
COLLADA (DAE)	Mesh and animation; another good option when FBX isn't available.
OBJ	Mesh only; this is a text format, so sometimes useful for streaming over the internet.
3DS	Mesh only; a pretty old and primitive model format.
DXF	Mesh only; a pretty old and primitive model format.
Maya	Works via FBX; requires this application to be installed.
3ds Max	Works via FBX; requires this application to be installed.
Blender	Works via FBX; requires this application to be installed.

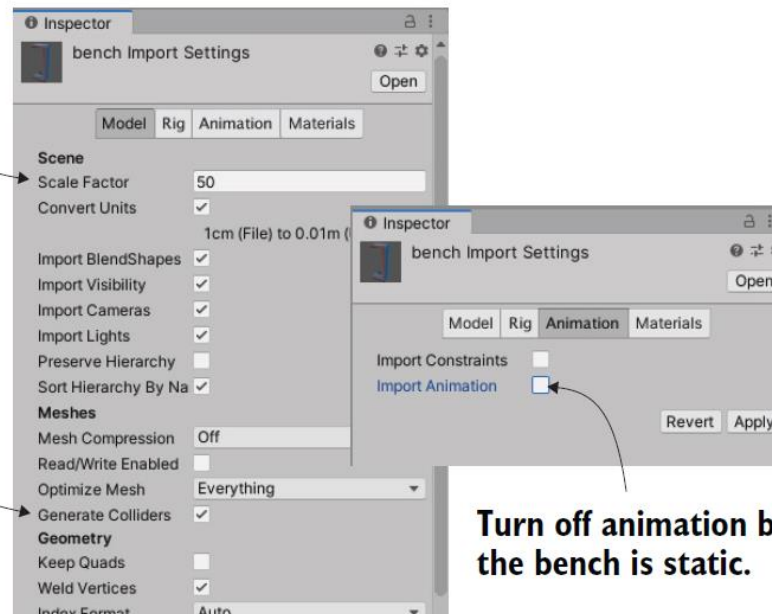
انتخاب یک گزینه به این بستگی دارد که آیا فایل از انیمیشن پشتیبانی می کند یا خیر. از آنجا که COLLADA و FBX تنها دو گزینه ای هستند که داده های انیمیشن را شامل می شوند، این دو گزینه برای انتخاب هستند. همه ابزارهای سه بعدی آن را به عنوان یک گزینه خروجی ندارند، خروجی FBX به بهترین شکل ممکن کار می کند، اما اگر از ابزاری بدون خروجی FBX استفاده می کنید، COLLADA نیز گزینه خوبی است. Blender از خروجی FBX پشتیبانی می کند، بنابراین ما از آن فرمت فایل استفاده خواهیم کرد.



- Drag the FBX file from the computer into Unity's Project
- You should change a few import settings immediately.
 - Change the Scale Factor to 50
 - click the Generate Colliders check box
 - Click Apply at the bottom after making these changes

**Default size is too small,
so set the scale to 50.**

**Optional: Generate a
collider or you can walk
through the bench.**



**Click Apply after making
changes to import settings.**

**Turn off animation because
the bench is static.**

Figure 4.15 Adjust import settings for the 3D model.



- Drag the image file from this project's scratch folder into Unity's Project
- New materials are often too shiny, so you may want to reduce the Smoothness setting to 0



This image relates to the model using "texture coordinates".

To understand the concept of texture coordinates, refer to the appendix.

مختصات بافت مدل برای تعریف این نگاشت تصویر به مش ویرایش شد.

علاوه بر تصاویر دو بعدی و مدل های سه بعدی، نوع باقی مانده محتوای بصری که هنرمندان بازی ایجاد می کنند یک سیستم ذرات است. تعریف در مقدمه این فصل توضیح داد که سیستم های ذرات مکانیسم های منظمی برای ایجاد و کنترل تعداد زیادی از اجسام متحرک هستند. سیستم های ذرات برای ایجاد جلوه های بصری مانند آتش، دود

- Create a new particle system
- From the GameObject menu, choose Effects > Particle System

Pause or reset the particle effect playing in the scene.

Click and drag the label **Playback Time** to play back and forth.

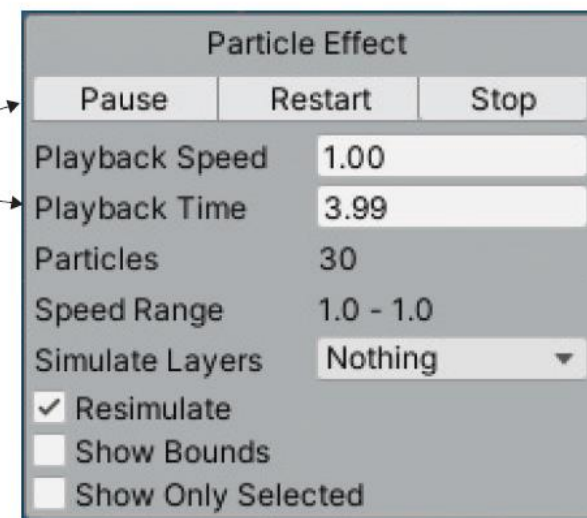
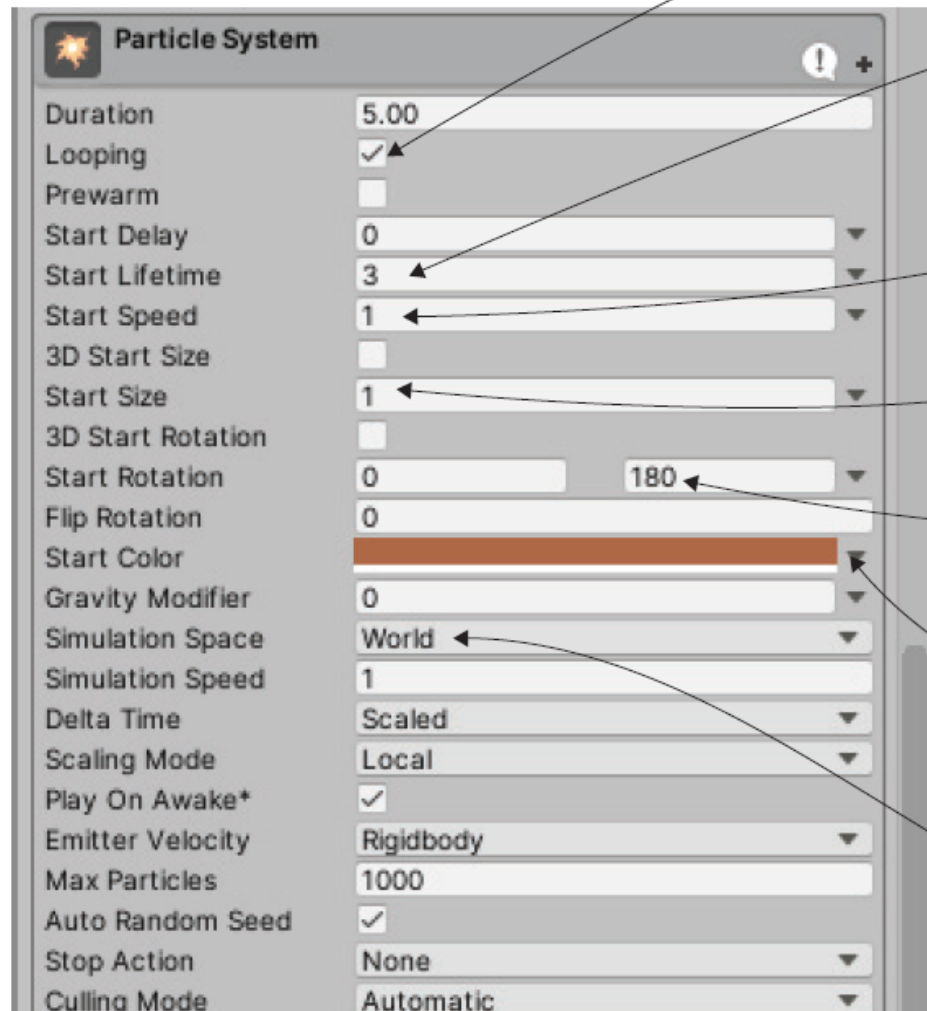


Figure 4.19 Playback panel for a particle system



Looping: The particle system keeps playing forever; leave the default.

Lifetime: How long the particle exists; reduce to 3.

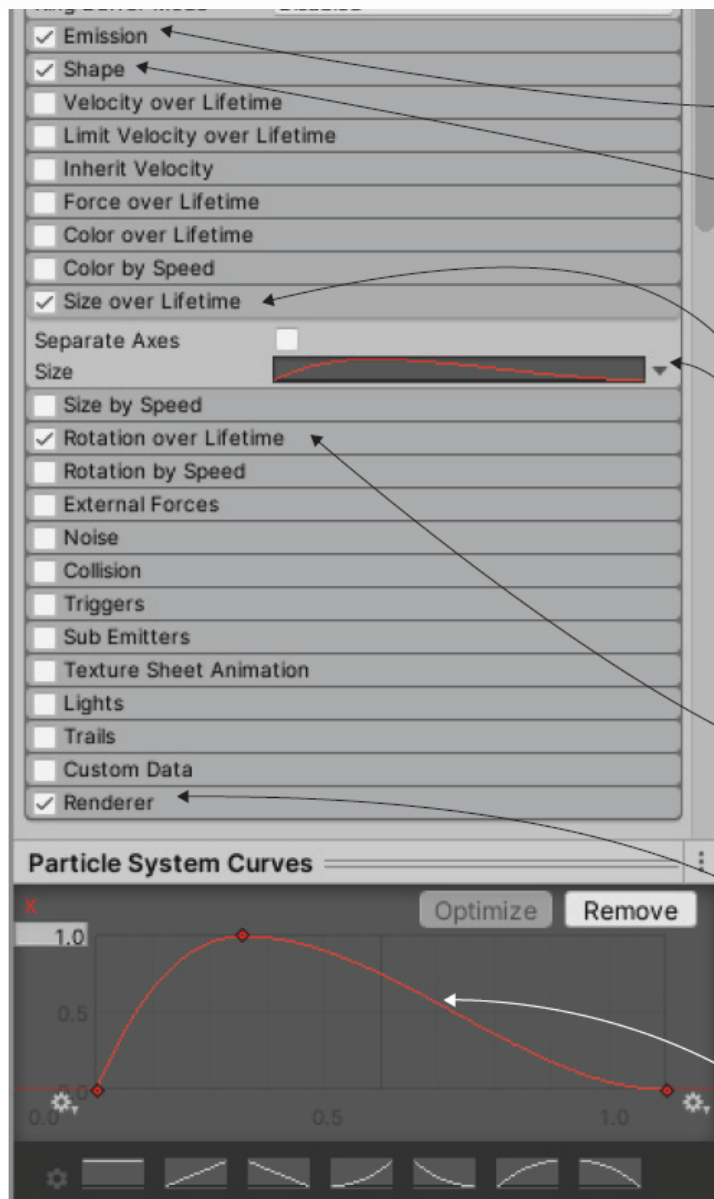
Speed: How fast the particle is moving; reduce to 1.

Size: How big the particle is; leave the default.

Rotation: The orientation of the particle; click the arrow menu to change to **Between Two Constants**, and set to 0 and 180.

Color: Tint the particles. We want a dim orange, like RGB values 182, 101, 58.

Local simulation space is fine for a still particle system, but **World** may be better for a system that's moving.



Emission: How quickly particles are emitted; leave the default.

Shape: The shape of the area emitted from. The default is a wide cone, but we want a small box to make a tight jet of flame (set to Box, and all numbers to 0.2)

Size over Lifetime: The particle grows and shrinks. The default is off; turn this on and click the arrow to set a curve. The bottom of this image shows a curve that quickly grows and then slowly shrinks back to 0.

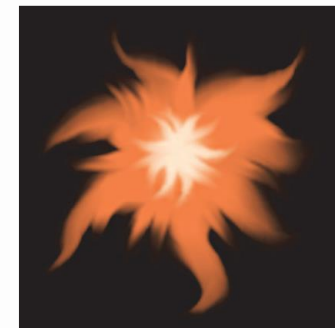
Rotation over Lifetime: The particle rotates while it moves. The default is off; turn this on and set Random Between to -80 and 80 to make different particles rotate in different directions.

Renderer: Set what each particle looks like. You could even set this to a mesh, but leave it as Billboard and drag a new material (explained shortly).

Select settings' curves (e.g., Size over Lifetime)

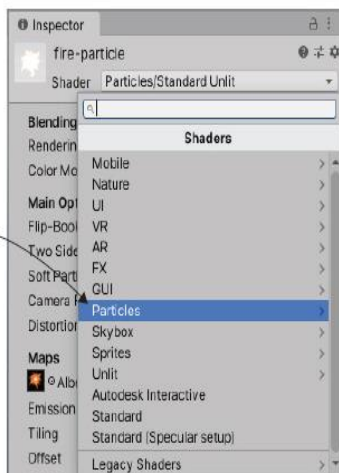
Applying a new texture for fire

- Import the image file into Unity. You add the texture to a material and apply that material to the particle system.
- Create a new material. Drag the fire image from Project up to the texture slot
- Click the Shader menu near the top of the material settings to see the list of available shaders. Instead of the standard default, a material for particles needs one of the shaders under the Particles submenu.
- Now switch the material's Rendering Mode to Additive.
- Select the particle system, expand Renderer at the bottom of the settings, and drag the material onto the Material slot



گزینه Additive باعث می شود پیکسل های مشکی حذف شوند.

From the material's
Shader menu, choose
Particles > Standard Unlit.



After choosing the particle shader,
change Rendering Mode to Additive.

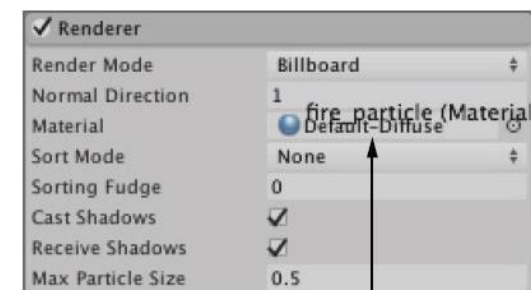
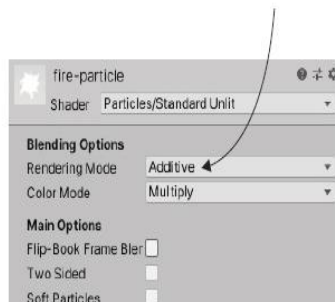
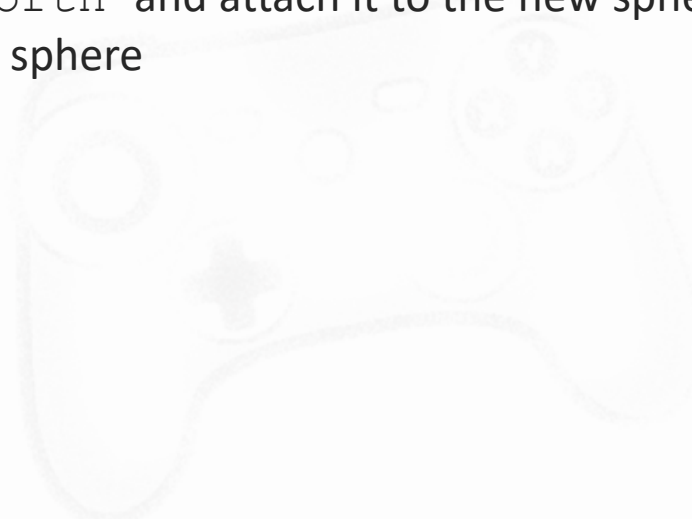


Figure 4.23 Setting the shader for the fire particle material

Figure 4.22 Assign a material to the particle system.



- Create a sphere (remember, GameObject > 3D Object > Sphere)
- Create a new script called `BackAndForth` and attach it to the new sphere
- make the particle system a child of the sphere
- Set the particle system at 0, 0, 0





```
public float speed = 3.0f;  
public float maxZ = 16.0f;  
public float minZ = -16.0f;  
private int direction = 1;
```

// Start is called before the first frame update

```
void Start()
```

```
{
```

```
}
```

// Update is called once per frame

```
void Update()
```

```
{
```

```
    transform.Translate(0, 0, direction * speed * Time.deltaTime);
```

```
    bool bounced = false;
```

```
    if (transform.position.z > maxZ || transform.position.z < minZ)
```

```
    {
```

```
        direction = -direction;
```

```
        bounced = true;
```

```
    }
```

```
    if (bounced)
```

```
    {
```

```
        transform.Translate(0, 0, direction * speed * Time.deltaTime);
```

```
    }
```

```
}
```

